The creation of the response uses `SprayJsonSupport._` and `DefaultJsonProtocol._`as before and then two JSON formatters, `jsonFormat1` and `jsonFormat2`:

```
import akka.http.scaladsl.marshallers.sprayjson.SprayJsonSupport._
import spray.json.DefaultJsonProtocol._

object HttpServerShardedContainer {
  ...

  implicit val cargoFormat: JsonFormat[Container.Cargo] =
      jsonFormat2(Container.Cargo)
  implicit val cargosFormat: RootJsonFormat[Container.Cargos] =
      jsonFormat1(Container.Cargos)
 ...
}
```

To serialize `Container.Cargos`, `jsonFormat1` handles its basic `List` type. But since it is a `List[Cargo]`, you also need to provide the formatter for `Cargo`: `jsonFormat2`, which has the two basic fields `String` and `Int`.

Unmarshalling the request

In this example, you use these formatters only for marshalling. You can also use them for unmarshalling when reading the request instead of using the parameters directive:

```
post { entity(as[Container.Cargo]) { cargo => ... }
```

This wouldn't work in the current example because you need the entity's ID to send the message to the entity, and the `Cargo` class doesn't have such a field. Nevertheless, the `entity` directive is very handy and often used.

With these formatters, you can create the following `get` to create a route at URL cargo?entityId=[myId], as follows: